

§ 6.7 深度优先搜索

6.7.1 策略

深度优先搜索（Depth-First Search, DFS）选取下一顶点的策略，可概括为：

优先选取最后一个被访问到的顶点的邻居

于是，以顶点 s 为基点的DFS搜索，将首先访问顶点 s ；再从 s 所有尚未访问到的邻居中任取其一，并以之为基点，递归地执行DFS搜索。故各顶点被访问到的次序，类似于树的先序遍历（5.4.2节）；而各顶点被访问完毕的次序，则类似于树的后序遍历（5.4.4节）。

6.7.2 实现

深度优先遍历算法可实现如代码6.4所示。

```

1 template <typename Tv, typename Te> //深度优先搜索DFS算法（全图）
2 void Graph<Tv, Te>::dfs(int s) { //assert: 0 <= s < n
3     reset(); int clock = 0; int v = s; //初始化
4     do //逐一检查所有顶点
5         if (UNDISCOVERED == status(v)) //一旦遇到尚未发现的顶点
6             DFS(v, clock); //即从该顶点出发启动一次DFS
7     while (s != (v = (++v % n))); //按序号检查，故不漏不重
8 }
9
10 template <typename Tv, typename Te> //深度优先搜索DFS算法（单个连通域）
11 void Graph<Tv, Te>::DFS(int v, int& clock) { //assert: 0 <= v < n
12     dTime(v) = ++clock; status(v) = DISCOVERED; //发现当前顶点v
13     for (int u = firstNbr(v); -1 < u; u = nextNbr(v, u)) //枚举v的所有邻居u
14         switch (status(u)) { //并视其状态分别处理
15             case UNDISCOVERED: //u尚未发现，意味着支撑树可在此拓展
16                 status(v, u) = TREE; parent(u) = v; DFS(u, clock); break;
17             case DISCOVERED: //u已被发现但尚未访问完毕，应属被后代指向的祖先
18                 status(v, u) = BACKWARD; break;
19             default: //u已访问完毕（VISITED，有向图），则视承袭关系分为前向边或跨边
20                 status(v, u) = (dTime(v) < dTime(u)) ? FORWARD : CROSS; break;
21         }
22     status(v) = VISITED; fTime(v) = ++clock; //至此，当前顶点v方告访问完毕
23 }
```

代码6.4 DFS算法

算法的实质功能，由子算法DFS()递归地完成。每一递归实例中，都先将当前节点 v 标记为DISCOVERED（已发现）状态，再逐一核对其各邻居 u 的状态并做相应处理。待其所有邻居均已处理完毕之后，将顶点 v 置为VISITED（访问完毕）状态，便可回溯。

若顶点 u 尚处于UNDISCOVERED（未发现）状态，则将边 (v, u) 归类为树边（tree edge），

并将 v 记作 u 的父节点。此后，便可将 u 作为当前顶点，继续递归地遍历。

若顶点 u 处于DISCOVERED状态，则意味着在此处发现一个有向环路。此时，在DFS遍历树中 u 必为 v 的祖先（习题[6-13]），故应将边 (v, u) 归类为后向边（back edge）。

这里为每个顶点 v 都记录了被发现和访问完成的时刻，对应的时间区间 $[dTime(v), fTime(v)]$ 均称作 v 的活跃期（active duration）。实际上，任意顶点 v 和 u 之间是否存在祖先/后代的“血缘”关系，完全取决于二者的活跃期是否相互包含（习题[6-12]）。

对于有向图，顶点 u 还可能处于VISITED状态。此时，只要比对 v 与 u 的活跃期，即可判定在DFS树中 v 是否为 u 的祖先。若是，则边 (v, u) 应归类为前向边（forward edge）；否则，二者必然来自相互独立的两个分支，边 (v, u) 应归类为跨边（cross edge）。

DFS(s)返回后，所有访问过的顶点通过parent[]指针依次联接，从整体上给出了顶点s所属连通或可达分量的一棵遍历树，称作深度优先搜索树或DFS树（DFS tree）。与BFS搜索一样，此时若还有其它的连通或可达分量，则可以其中任何顶点为基点，再次启动DFS搜索。

最终，经各次DFS搜索生成的一系列DFS树，构成了DFS森林（DFS forest）。

6.7.3 实例

图6.8针对含7个顶点和10条边的某有向图，给出了DFS搜索的详细过程。请留意观察顶点时间标签的设置，顶点状态的演变，边的分类和结果，以及DFS树（森林）的生长过程。

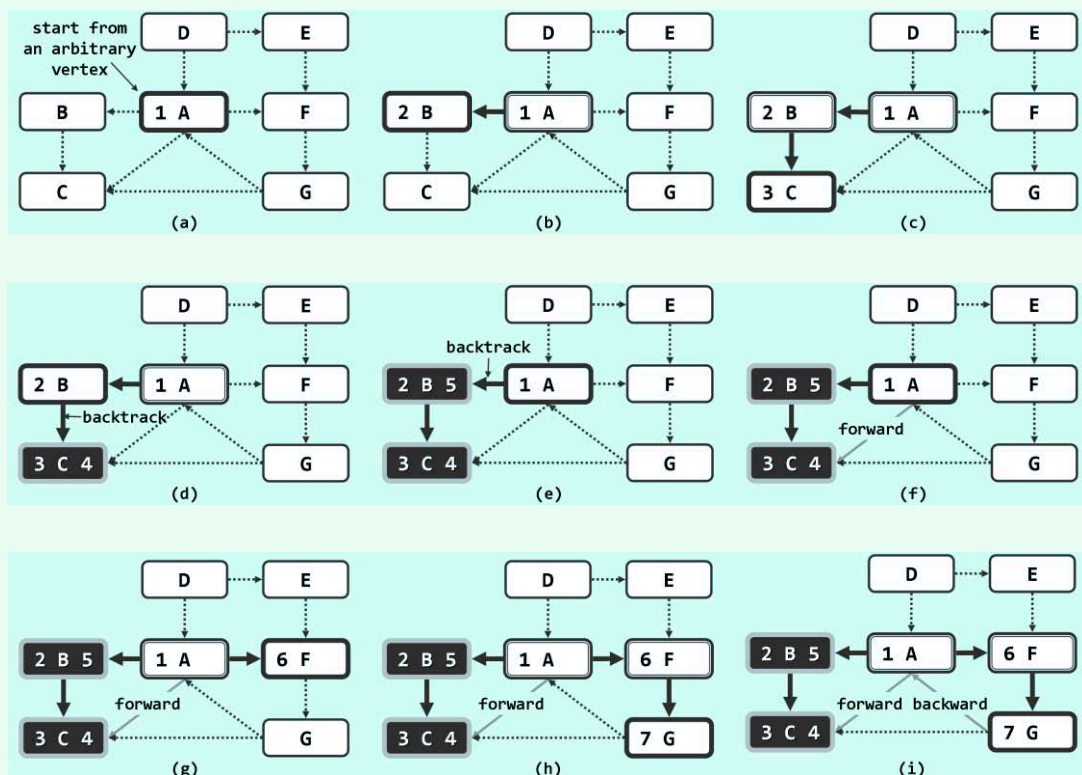


图6.8 深度优先搜索实例（粗边框白色，为当前顶点；细边框白色、双边框白色和黑色，分别为处于UNDISCOVERED、DISCOVERED和VISITED状态的顶点；dTime和fTime标签，分别标注于各顶点的左右）

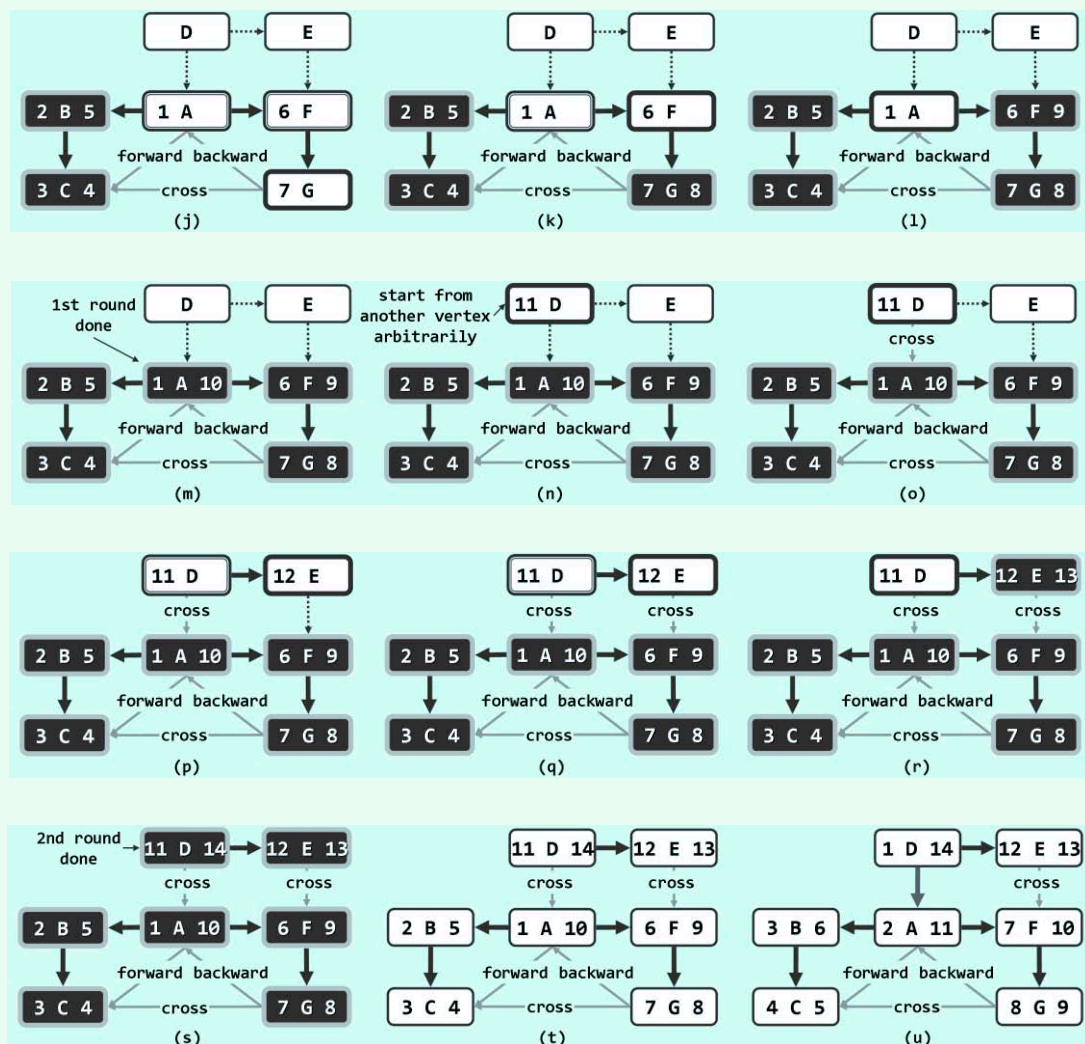


图6.8 深度优先搜索实例(续): (a~m)对应于DFS(A), (n~s)为随后的DFS(D)

最终结果如图(t)所示,为包含两棵DFS树的一个DFS森林。可以看出,选用不同的起始基点,生成的DFS树(森林)也可能各异。如本例中,若从D开始搜索,则DFS森林可能如图(u)所示。

图6.9以时间为横坐标,绘出了图6.8(u)中DFS树内各顶点的活跃期。可以清晰地看出,活跃期相互包含的顶点,在DFS树中都是“祖先-后代”关系(比如B之于C,或者D之于F);反之亦然。

这种对应关系并非偶然,籍此可以便捷地判定节点之间的承袭关系(习题[6-12])。故无论是对DFS搜索本身,还是对基于DFS的各种算法而言,时间标签都至关重要。

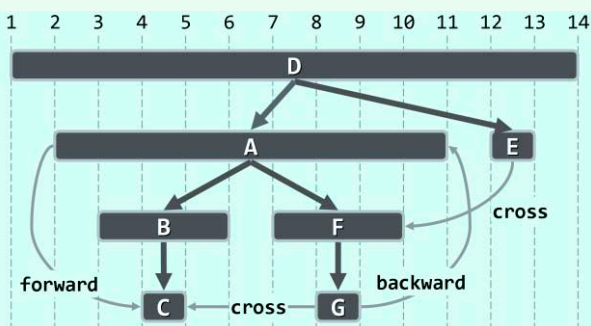


图6.9 活跃期与“祖先-后代”关系之间的对应关系

6.7.4 复杂度

除了原图本身,深度优先搜索算法所使用的空间,主要消耗于各顶点的时间标签和状态标记,以及各边的分类标记,二者累计不超过 $O(n) + O(e) = O(n + e)$ 。当然,如采用以上代码6.4的直接递归实现方式,操作系统为维护运行栈还需耗费一定量的空间——尽管这部分增量在渐进意义下还不足以动摇以上结论。为此,不妨仿照5.4节的做法,通过显式地引入并维护一个栈结构,将DFS算法改写为迭代版本(习题[6-14])。

时间方面,首先需要花费 $O(n + e)$ 时间对所有顶点和边的状态复位。不计对子函数DFS()的调用,dfs()本身对所有顶点的枚举共需 $O(n)$ 时间。不计DFS()之间相互的递归调用,每个顶点、每条边只在子函数DFS()的某一递归实例中耗费 $O(1)$ 时间,故累计亦不过 $O(n + e)$ 时间。综合而言,深度优先搜索算法也可在 $O(n + e)$ 时间内完成。

6.7.5 应用

深度优先搜索无疑是最为重要的图遍历算法。基于DFS的框架,可以导出和建立大量的图算法。以4.4节英雄忒修斯营救公主的故事为例,为寻找从迷宫入口(起始顶点)至公主所在位置(目标顶点)的通路,可将迷宫内不同位置之间的联接关系表示为一幅图,并将问题转化为起点和终点之间的可达性判定,从而可利用DFS算法便捷地加以解决。非但如此,一旦找到通路,则不仅可以顺利抵达终点与公主会合,还能沿这条通路安全返回。当然,与广度优先搜索一样,深度优先搜索也可用作连通分量的分解,或者有向无环图的判定。

下面仅以拓扑排序和双连通域分解为例,对DFS模式的应用做更为具体的介绍。

§ 6.8 拓扑排序

6.8.1 应用

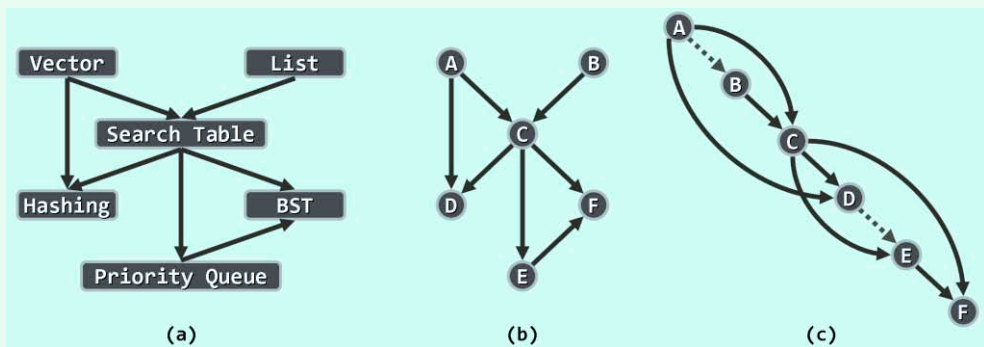


图6.10 拓扑排序

以教材的编写这一实际问题为例。首先,作者可借助有向图结构,整理出相关知识点之间的依赖关系。如图6.10(a)所示,因向量是散列表和查找表的基础知识点,故从Vector发出两条边分别指向Hashing和Search Table;同理,查找表是二叉搜索树的基础知识点,故也从前者引出一条边指向后者;...;诸如此类。那么,如何将这些知识点串联为一份教学计划,以保证在整个授课进程中,每堂课的基础知识点均在此前业已讲授呢?